

Interview Q&A: Swiggy Delivery Time Predictor

Project: Swiggy Delivery Time Predictor | Bharat Nirman Lab, IIT Kanpur | Summer Intern (Jun'26 – Aug'26)

Key claims on resume: 17% missing data handled via KNN Imputer · Haversine spatial features · ColumnTransformer pipelines · LightGBM, XGBoost, Random Forest, SVR vs Linear, Ridge, Lasso · Optuna tuning · LightGBM MAE 3.01 min, R^2 0.83 · RF MAE 3.28, R^2 0.80 · Linear MAE 4.70, R^2 0.60 · 36% error reduction

How to use this doc: Answers are written as what you would most likely say. Anything marked *(your value)* must be replaced with what you actually did. Interviewers can tell when a claim is not yours. Part A and B are core questions. Part C onwards is the deep dive.

PART A: CORE QUESTIONS

A1. Project Overview

Q1. Walk me through your project in 1 minute. I built a regression model to predict food delivery time for Swiggy-style orders. I cleaned the data, imputed 17% missing values with KNN, engineered distance features using Haversine, and built ColumnTransformer pipelines. I compared 7 models, tuned them with Optuna, and LightGBM was best (MAE 3.01 min, R^2 0.83), a 36% error reduction over Linear Regression.

Q2. What was the business problem? Accurate ETAs improve customer trust, rider allocation, and kitchen/dispatch planning. Wrong ETAs cause cancellations and bad ratings.

Q3. Why is this regression and not classification? The target (delivery time in minutes) is continuous.

Q4. What was your role versus the mentor's? I did end-to-end modeling: EDA, preprocessing, feature engineering, training, tuning. The mentor guided direction and reviewed results.

Q5. What was the target variable? Delivery time in minutes.

Q6. Who would use this model? The customer app (ETA display), the dispatch system, and ops teams.

Q7. Hardest part of the project? Handling missing data and designing spatial and temporal features that actually improved the model.

Q8. If you had one more month? Add real-time traffic and weather, quantile regression for ETA ranges, and proper error analysis by city and time slot.

A2. Data

Q9. How big was the dataset? *(your rows and columns)*

Q10. What features did it have? Geographic (restaurant and delivery lat/long), temporal (order time, day, hour), and order-specific (rider age and ratings, order type, vehicle type, multiple deliveries, etc.). *(Adjust to yours.)*

Q11. Where did the data come from? *(Be honest: Kaggle, company-provided, or scraped.)*

Q12. Was the target skewed? Delivery times are usually right-skewed. Trees handle that fine; linear models may benefit from a log transform

Q13. Did you find outliers? Yes, in distance (bad GPS such as (0,0) or outside India) and in delivery time. I used domain-based filters.

Q14. Duplicates? Checked and removed.

Q15. Inconsistent categories or data types? Fixed trailing spaces, mixed casing, and "NaN" stored as text.

Q16. Is data balanced across cities and time slots? (*Check.*) If not, performance may be worse on rare segments.

Q17. Label noise? Yes. Real delivery time includes rider delay, restaurant wait, and customer unavailability, which aren't fully observed.

A3. Missing Values and KNN Imputer

Q18. How did you find 17% missing? `df.isnull().mean()` plus heatmaps or missingno.

Q19. Why KNN Imputer over mean/median? Mean/median ignore relationships between features and shrink variance. KNN uses similar rows, preserving relationships.

Q20. How does KNN Imputer work? For a row with a missing value, find the k nearest rows using distance over the non-missing features, then average those neighbors' values for the missing one.

Q21. What k and why? (*your value*) Typically 5. Small k is noisy, large k oversmooths. Ideally tuned by masking known values and measuring error.

Q22. Does it need scaling? Yes, it's distance-based; large-scale features dominate otherwise.

Q23. Can it handle categoricals? Not directly. Encode first, or impute categoricals with mode separately.

Q24. Fit before or after the split? After, on train only. Fitting on full data leaks test information. (*If you did it before, admit it and say how you'd fix it.*)

Q25. Drawbacks? Slow on big data ($O(n^2)$), sensitive to scaling and k, weak in high dimensions.

Q26. MCAR, MAR, MNAR? MCAR: missing completely at random. MAR: depends on observed variables. MNAR: depends on the missing value itself. KNN works best for MCAR and MAR.

Q27. Which was yours? (*Say what you analyzed.*) Likely MAR or MCAR; not formally tested.

Q28. Why not drop rows? Losing 17% hurts data volume and can bias results if missingness isn't random.

Q29. IterativeImputer (MICE)? Models each feature from the others, often more accurate but slower. KNN was a good tradeoff.

Q30. Could you skip imputation? For LightGBM and XGBoost yes, they handle NaN natively. Linear, Ridge, Lasso, SVR can't, so imputation was needed for a fair comparison.

Q31. How did you validate imputation? Mask known values, impute, compare with truth (RMSE); or compare distributions before and after.

Q32. Missing indicator flags? Optional but useful, since missingness itself can be predictive.

A4. Feature Engineering and Haversine

Q33. What is Haversine? Great-circle distance between two points on a sphere from lat/long.

Q34. Formula? $a = \sin^2(\Delta\phi/2) + \cos \phi_1 \cdot \cos \phi_2 \cdot \sin^2(\Delta\lambda/2)$; $c = 2 \cdot \text{atan2}(\sqrt{a}, \sqrt{1-a})$; $d = R \cdot c$, $R \approx 6371$ km.

Q35. Why not Euclidean on lat/long? Longitude degrees shrink away from the equator; Euclidean ignores curvature and gives degrees, not km.

Q36. Limitations? Straight-line, not road distance; ignores roads, traffic, one-ways.

Q37. Alternatives? Manhattan, OSRM/Google road distance, Vincenty/geodesic.

Q38. Why is it accurate enough here? Over short city distances the spherical error is tiny versus the road-vs-straight-line gap.

Q39. Other features? Hour, day of week, weekend flag, rush-hour flag, time buckets, prep time. (*Edit to yours.*)

Q40. How did you extract time features? `pd.to_datetime` and `.dt.hour`, `.dt.dayofweek`, etc.

Q41. Cyclical time encoding? $\sin/\cos$ so hour 23 and 0 are adjacent. (*Say if you did it.*)

Q42. Most important engineered feature? (*Check importance.*) Usually distance.

Q43. Multicollinearity? Checked via correlation or VIF. Matters for linear models, not trees.

Q44. Feature selection? (*Yes/no.*) Options: drop low importance, Lasso zeroing, RFE.

Q45. Raw lat/long as features? Used for distance; raw coordinates can also capture area-specific effects.

A5. Preprocessing and ColumnTransformer

Q46. What is ColumnTransformer? Applies different preprocessing to different column groups in one object.

Q47. Why pipelines? Prevents leakage, guarantees identical train/test transforms, cleaner code, easier deployment.

Q48. What does "dynamic" mean? Columns picked by dtype automatically via `make_column_selector`, not hardcoded names.

Q49. Which categorical encoder? One-Hot for low cardinality. (*Mention others if used.*)

Q50. One-Hot vs Label vs Target encoding? One-Hot: no false order, high dimensionality. Label: compact but implies order. Target: handles high cardinality but leaks unless done within CV folds.

Q51. Unseen categories at inference? `OneHotEncoder(handle_unknown='ignore')`.

Q52. Which scaler? `StandardScaler`; needed for SVR, linear, Ridge, Lasso, KNN.

Q53. Do trees need scaling? No, splits are threshold-based.

Q54. Standard vs MinMax vs Robust? Standard: zero mean, unit variance. MinMax: $[0,1]$, outlier-sensitive. Robust: median and IQR, outlier-resistant.

Q55. One pipeline for all models? Same preprocessing for fairness; scaling is harmless for trees.

Q56. What is data leakage? Where could it happen? Test information influencing training: imputation/scaling/encoding fit before split, or target-derived features.

Q57. How did you split? (*Your split.*) Typically 80/20 random; time-based would be better.

Q58. Why is a random split problematic? Nearby-in-time or space orders land in both sets and inflate scores.

A6. Models

Q59. Why start with Linear Regression? Simple, interpretable baseline.

Q60. Assumptions? Linearity, independence, homoscedasticity, normal residuals, no multicollinearity.

Q61. Why MAE 4.70 for linear? Non-linearities and interactions (distance $\times$ traffic $\times$ hour) that linear can't capture.

Q62. Ridge vs Lasso? Ridge: L2, shrinks. Lasso: L1, can zero out coefficients. ElasticNet: both.

Q63. When does Lasso beat Ridge? Many irrelevant features, you want sparsity.

Q64. How did you tune alpha? Optuna or CV (RidgeCV, LassoCV).

Q65. Why didn't regularization help? The issue was underfitting (bias), not variance.

Q66. How does Random Forest work? Many trees on bootstrap samples with random feature subsets, averaged.

Q67. Why better than one tree? Averaging reduces variance.

Q68. Key RF hyperparameters? `n_estimators`, `max_depth`, `min_samples_split`, `min_samples_leaf`, `max_features`.

Q69. OOB score? Validation score on samples left out of each tree's bootstrap.

Q70. Why did LightGBM beat RF? Boosting reduces bias sequentially; RF only reduces variance.

Q71. How does SVR work? Fits within an ϵ -tube; errors inside ϵ are ignored; outside penalized via C .

Q72. SVR hyperparameters? C , epsilon, kernel, gamma.

Q73. Why scaling for SVR? Kernel distances get distorted otherwise.

Q74. Why SVR underperformed? Poor scaling on large data ($O(n^2-n^3)$), sensitive to hyperparameters, possibly under-tuned.

Q75. Kernel trick? Computes inner products in a high-dimensional space without mapping explicitly.

Q76. How does gradient boosting work? Trees built sequentially, each fitting the residuals (gradients) of the ensemble so far.

Q77. XGBoost vs classical GBM? L1/L2 regularization, second-order gradients, parallel split finding, native missing handling.

Q78. What's different in LightGBM? Leaf-wise growth, histogram binning, GOSS, EFB.

Q79. Leaf-wise vs level-wise? Level-wise splits all nodes at a depth. Leaf-wise splits the leaf with the largest loss reduction: lower loss, but can overfit on small data.

Q80. Params that control overfitting in LightGBM? `num_leaves`, `max_depth`, `min_child_samples`, `lambda_l1/l2`, `feature_fraction`, `bagging_fraction`.

Q81. Why is num_leaves key? Primary complexity control in leaf-wise trees; keep below $2^{\text{max_depth}}$.

Q82. Why LightGBM best here? Captures non-linear interactions among spatial, temporal, and order features; strong on tabular data; well tuned.

Q83. Why did XGBoost lose? (*Your numbers.*) Gap probably small; could be tuning or noise.

Q84. Learning rate tradeoff? Low rate needs more trees but generalizes better.

Q85. Early stopping? Stop when the validation metric stops improving.

Q86. Bagging vs boosting? Bagging: parallel, reduces variance. Boosting: sequential, reduces bias.

Q87. LightGBM categoricals? Native via `categorical_feature`, no One-Hot needed.

Q88. Did you use native handling? (*Yes/no.*) If not, mention it as an improvement.

A7. Optuna

Q89. What is Optuna? Hyperparameter optimization framework using TPE (Bayesian-style) with pruning.

Q90. Why over Grid/Random search? Grid is exhaustive, random is blind; Optuna learns from past trials and prunes bad ones.

Q91. How does TPE work? Models distributions of good vs bad hyperparameters and samples where the good/bad ratio is high.

Q92. Objective? Minimize cross-validated MAE (or RMSE).

Q93. Trials? (*Your number.*) Typically 50–100.

Q94. What did you tune for LightGBM? E.g. `num_leaves` 20–150, `learning_rate` 0.01–0.3 (log), `n_estimators` 100–1000, `min_child_samples` 5–100, `subsample`, `colsample_bytree`, `reg_alpha`, `reg_lambda`. (*Edit to yours.*)

Q95. Why log-scale for learning rate? Its effect is multiplicative; 0.01 vs 0.1 matters far more than 0.21 vs 0.3.

Q96. Tuned on test set? No, CV on train; test kept for final evaluation.

Q97. Avoiding overfitting to validation during tuning? k-fold CV averaging plus an untouched test set.

Q98. Pruning? (*Yes/no.*) MedianPruner or HyperbandPruner.

Q99. Improvement from tuning? (*Your before/after numbers.*)

Q100. Was the model comparison fair? Same split, same preprocessing, same CV folds, similar trial budget per model.

A8. Metrics

Q101. MAE? Mean absolute error, in minutes.

Q102. MAE 3.01 meaning? Predictions off by about 3 minutes on average.

Q103. R^2 ? $1 - SS_{res}/SS_{tot}$; variance explained.

Q104. R^2 0.83? Explains 83% of the variance in delivery time.

Q105. Why both? MAE is interpretable in minutes; R^2 is relative to a mean-predictor baseline.

Q106. MAE vs RMSE? MAE is robust to outliers; RMSE penalizes large errors more.

Q107. Which matters more? MAE for customer-facing interpretability; RMSE if big misses are costly.

Q108. Can R^2 be negative? Yes, when worse than predicting the mean.

Q109. Adjusted R^2 ? R^2 penalized for number of predictors.

Q110. How is the 36% computed? $(4.70 - 3.01)/4.70 \approx 36\%$, relative MAE reduction vs Linear Regression.

Q111. Isn't a weak baseline misleading? Fair point. Also compare against a mean/median baseline and RF ($3.28 \rightarrow 3.01 \approx 8\%$).

Q112. Is 0.23 MAE gap vs RF significant? Not necessarily; check fold variance and a paired test.

Q113. Train or test set? Held-out test. (*Confirm.*)

Q114. Train vs test check? Yes, a big gap signals overfitting.

Q115. MAPE? Useful for relative error but explodes when actuals are small.

Q116. Is 3 min good enough? Depends on mean delivery time; if ~ 25 min, that's $\sim 12\%$ error, decent for ETA.

A9. Validation

Q117. Cross-validation? Train on $k-1$ folds, validate on 1, average over folds.

Q118. Folds? (*Your number.*) Usually 5.

Q119. Bias-variance tradeoff? High bias: underfit (linear here). High variance: overfit (deep trees). Aim for balance.

Q120. Detecting overfitting? Train vs validation gap, learning curves.

Q121. Preventing it? Regularization, early stopping, depth/leaf limits, subsampling, CV.

Q122. Time-based CV? `TimeSeriesSplit`, avoids training on the future.

Q123. Distribution shift? Performance degrades; monitor drift and retrain.

A10. Interpretability and Error Analysis

Q124. Top features? (*Yours.*) Typically distance, hour/rush hour, rider features.

Q125. How computed? LightGBM built-in gain/split; SHAP or permutation importance are more reliable.

Q126. Why built-in can mislead? Split-count importance is biased toward continuous or high-cardinality features.

Q127. SHAP? Game-theoretic per-prediction feature contributions.

Q128. Permutation importance? Shuffle a feature, measure the performance drop.

Q129. Largest errors? Likely long distances, peak hours, bad weather. (*Verify.*)

Q130. Residual plots? Residual vs predicted to check bias and heteroscedasticity.

Q131. Under or over-predict? (*Check mean residual.*) Under-prediction usually hurts customers more.

Q132. Error analysis approach? Slice by city, hour, distance bucket, order type; find high-MAE segments; engineer for them.

A11. Deployment

Q133. **How to deploy?** Save the full pipeline with joblib, serve via FastAPI/Flask, containerize with Docker.

Q134. **Why save the whole pipeline?** Same preprocessing at inference, avoids train-serve skew.

Q135. **Train-serve skew?** Features computed differently in training vs production.

Q136. **Latency?** LightGBM is fast; KNN Imputer at inference can be slow, so precompute or use a simpler imputer.

Q137. **Monitoring?** Rolling MAE, input drift (PSI, KS), prediction distribution.

Q138. **When to retrain?** Scheduled or on drift/performance drop.

Q139. **New restaurants or cities?** Cold start: fall back to general features, retrain, geographic similarity.

Q140. **Features unavailable at prediction time?** Anything known only post-delivery (actual prep time, actual route). Using them is leakage.

Q141. **Leakage risk in your features?** Confirmed all were available at order placement. (*Verify.*)

A12. Curveballs

Q142. **Why R^2 0.83 and not higher?** Unobserved traffic, weather, rider behavior, restaurant delays.

Q143. **How to push further?** Real-time traffic/weather, road distance, restaurant historical prep time, rider history, stacking.

Q144. **Ensemble or stacking?** Likely small gains by blending LightGBM, XGBoost, RF.

Q145. **Deep learning?** On moderate tabular data boosting usually matches or beats it; DL helps for ID embeddings or route sequences.

Q146. **Give a range instead of a point?** Quantile regression (`objective='quantile'`) or conformal prediction.

Q147. **Asymmetric error cost?** Custom or quantile loss penalizing under-prediction more.

Q148. **Why not predict log(time)?** Could help for skew; exponentiate predictions back.

Q149. **How do you know it generalizes?** I don't fully; validate on new periods and cities.

Q150. **Statistically significant?** Not formally; use repeated CV with paired t-test or Wilcoxon.

Q151. **"KNN Imputer is a bad idea."** Slow and distance-sensitive; settle it empirically against median imputation and native NaN handling.

Q152. **Remove the imputer for LightGBM?** Probably small change; run the ablation.

Q153. **Ablation studies?** Ideally remove Haversine, imputation, time features one at a time.

Q154. **How do you know Haversine helped?** With vs without comparison. (*Give numbers.*)

Q155. **Limitations?** No traffic/weather, straight-line distance, possibly random split, limited generalization testing, no deployment.

Q156. **Production-ready?** Not yet: time-based validation, monitoring, latency checks, edge cases.

Q157. **What would you do differently?** Time-based split, ablations, SHAP, fuller error analysis.

Q157. **What would you do differently?** Time-based split, ablations, SHAP, outlier error analysis.

A13. Resume-Specific

Q158. **"Engineered spatial features"**, what else besides Haversine? (*Bearing, Manhattan, zone/cluster. Claim only what you did.*)

Q159. **Why SVR on large tabular data?** To cover different model families for a fair comparison.

Q160. **What does "advanced models" mean?** Non-linear models capturing interactions, versus the linear baseline family.

Q161. **How did you decide LightGBM was best?** Lowest MAE, highest R^2 on held-out test, confirmed with CV.

Q162. **Is "36%" fair?** It's relative MAE reduction vs Linear Regression; I state the baseline whenever I mention it.

Q163. **What did you learn?** End-to-end ML workflow, leakage-free pipelines, tuning with Optuna.

A14. Quick-Fire Fundamentals

Q164. **Parameters vs hyperparameters?** Learned from data vs set before training.

Q165. **Regularization?** Penalty on complexity to reduce overfitting.

Q166. **L1 vs L2?** L1 gives sparsity; L2 shrinks smoothly.

Q167. **Curse of dimensionality?** Distances lose meaning and data gets sparse as dimensions grow.

Q168. **Gradient descent?** Move parameters opposite the loss gradient.

Q169. **Regression tree split criterion?** Variance (MSE) reduction.

Q170. **Trees robust to feature outliers?** Splits depend on order, not magnitude.

Q171. **fit_transform vs transform?** Learn and apply (train) vs apply learned parameters (test).

Q172. **fit vs predict?** Train vs generate outputs.

Q173. **random_state?** Reproducibility.

Q174. **Stratified split for regression?** Not directly; bin the target first.

Q175. **Interpolation vs extrapolation in trees?** Trees can't extrapolate; predictions flatten at the extremes.

PART B: DEEP DIVE

B1. Numbers and Consistency Checks (interviewers love these)

Q176. R^2 is 0.83 and MAE is 3.01. **Can you estimate the RMSE and the target's standard deviation?** Not exactly, but roughly. RMSE is always $\geq$ MAE, so $RMSE \geq 3.01$. If errors are roughly normal, $RMSE \approx 1.25 \times MAE \approx 3.8$. Since $R^2 = 1 - MSE/Var(y)$, $Var(y) \approx MSE/0.17 \approx 14.4/0.17 \approx 85$, so $std \approx 9.2$ min. (*Compare with your real std; if they don't match, say the errors are heavy-tailed.*)

Q177. **Linear has R^2 0.60 and MAE 4.70. Is that consistent with LightGBM's numbers?** Yes. MSE scales with $(1 - R^2)$: 0.40 vs 0.17, so linear MSE is about $2.35 \times$ LightGBM's. MAE ratio is $4.70/3.01 \approx 1.56$, and $\sqrt{2.35} \approx 1.53$.

They line up, which is a sanity check that the numbers are internally consistent.

Q178. Why is the R^2 gap (0.60 $\rightarrow$ 0.83) so large but MAE gap is "only" 36%? R^2 is on squared error, so it amplifies differences. MAE is linear. A 36% MAE reduction corresponds to roughly 58% reduction in MSE, which matches R^2 moving from 0.60 to 0.83 (0.40 $\rightarrow$ 0.17 is a 57.5% cut).

Q179. What is the MAE of a model that always predicts the mean? The median? For the mean predictor $R^2 = 0$ and $MAE \approx 0.8 \times \text{std} \approx 7.4 \text{ min}$ (if roughly normal). The median predictor minimizes MAE among constants, slightly lower. So 3.01 is about a 60% improvement over the naive baseline. (*Compute the real value.*)

Q180. If your test set has N rows, what is the uncertainty on MAE 3.01? Standard error $\approx \text{std}(|\text{error}|)/\sqrt{N}$. With, say, 10,000 rows and std of absolute error ~ 2.5 , $SE \approx 0.025$, so the 0.23 gap vs RF is many SEs wide. But errors are correlated, so use bootstrap CIs rather than trusting this blindly.

Q181. Why does the resume report MAE but you tuned for something else (or the same)? (*Be consistent.*) Say the metric you optimized in Optuna. Optimizing RMSE but reporting MAE is fine if you explain it. Best practice is to tune on the metric you report.

Q182. Is the 36% reduction relative to Linear or to the mean baseline? Linear Regression. $(4.70 - 3.01) / 4.70 = 35.96\%$. Against the mean baseline it'd be larger, but I chose the more honest, stronger baseline.

Q183. Where are the Ridge, Lasso, XGBoost, SVR numbers? (*Have them ready.*) Typically Ridge and Lasso $\approx$ Linear (4.7ish), SVR in between, XGBoost close to LightGBM. If you can't recall, you look unprepared. Keep a results table in your notes.

B2. KNN Imputer Internals

Q184. What distance does sklearn's KNNImputer use? `nan_euclidean`: Euclidean distance computed over the coordinates present in both rows, then scaled up by `total_features/present_features` to compensate for missing coordinates.

Q185. Write the nan_euclidean formula. $\text{dist} = \sqrt{(\text{n_total} / \text{n_present}) \times \sum \text{over present coords } (x_i - y_i)^2}$.

Q186. What does `weights='distance'` do? Neighbors are weighted by inverse distance, so closer neighbors contribute more. Default is `'uniform'`.

Q187. What happens if a neighbor also has the feature missing? The imputer only uses neighbors that have a value for that feature, and still takes the k nearest among those.

Q188. What happens if a whole column is missing? That column is dropped (or kept with `keep_empty_features=True`). It can't be imputed from neighbors.

Q189. Time complexity? $O(\text{n_missing_rows} \times \text{n_train_rows} \times d)$. It's brute-force; no tree index. On millions of rows it is impractical.

Q190. How do you make KNN imputation faster? Subsample the reference set, impute in chunks, reduce dimensions, or use a simpler imputer for large columns. Alternatively use LightGBM's native NaN handling.

Q191. Which columns did you feed to KNN Imputer, all of them? (*Your answer.*) Irrelevant columns add noise to the distance. Better to use a relevant subset of numeric features.

Q192. Does imputing the target make sense? No. Never impute the target; drop rows with missing target.

Q192. Does imputing the target make sense? NO. Never impute the target, drop rows with missing target.

Q193. Does imputation reduce variance, and does it matter? Yes. Imputed values are averages, so variance shrinks and correlations inflate. This can make standard errors too small, but for pure prediction it matters less.

Q194. Single vs multiple imputation? Single (KNN, mean) gives one filled dataset. Multiple imputation creates several plausible datasets and pools results, capturing imputation uncertainty. For prediction, single imputation is typical.

Q195. Where in the pipeline should KNNImputer sit with respect to encoding and scaling? Numeric branch: scale → impute (so distances aren't dominated by large-scale features) → optionally scale again. Categoricals: impute with mode, then One-Hot. All inside ColumnTransformer so it's fit only on training folds.

Q196. If missing values were in categorical columns, what then?

SimpleImputer(strategy='most_frequent') or a new "Unknown" category, which often works best for trees.

Q197. Suppose missingness correlates with delivery time (MNAR). What breaks? Imputation will bias estimates because the pattern carrying the signal is erased. Adding missing-indicator columns recovers that signal.

B3. Haversine and Geo Details

Q198. Derive the intuition behind Haversine. It uses the haversine function $\text{hav}(\theta) = \sin^2(\theta/2)$ to avoid the numerical instability of the spherical law of cosines for small distances.

Q199. Why is Haversine numerically better than the spherical law of cosines? For very small distances, cos of nearly identical angles loses precision. Haversine stays stable.

Q200. Error of Haversine vs true geodesic? Up to ~0.3–0.5% because Earth is an ellipsoid, not a sphere. Irrelevant for intra-city delivery.

Q201. Write vectorized Haversine in NumPy.

```
import numpy as np

def haversine(lat1, lon1, lat2, lon2):
    R = 6371.0
    lat1, lon1, lat2, lon2 = map(np.radians, [lat1, lon1, lat2, lon2])
    dlat, dlon = lat2 - lat1, lon2 - lon1
    a = np.sin(dlat/2)**2 + np.cos(lat1) * np.cos(lat2) * np.sin(dlon/2)**2
    return 2 * R * np.arcsin(np.sqrt(a))
```

Q202. Why convert to radians? Trig functions in NumPy expect radians.

Q203. Units of your distance feature? Kilometres.

Q204. How do you handle bad coordinates? Filter nulls, zeros, out-of-range values ($\text{lat} \in [-90, 90]$, $\text{lon} \in [-180, 180]$), and points outside India's bounding box. Also check for lat/long swapped columns, and negative coordinates caused by sign errors.

Q205. How would you detect distance outliers? Look at the distribution of distance; anything beyond a realistic delivery radius (say > 50 km) is suspect. Also check distance versus delivery time scatter.

Q206. Why might delivery time vs distance not be linear? Fixed overheads (prep, pickup, handoff) dominate short trips, speed varies by traffic, and long trips use faster roads. Gives an S-shaped or saturating relationship, hence trees beat linear.

Q207. Can you do better with spatial features than distance? Yes: geohash or grid-cell of restaurant and customer, distance to city centre, direction (bearing), clusters via KMeans, and neighborhood-level historical delivery stats.

Q208. What is bearing and why use it? The compass direction from restaurant to customer. It can capture directional effects like one-way arterial roads.

Q209. How would you use road-network distance? Query OSRM or Google Distance Matrix for each pair. It's costly at scale, so cache or approximate with Manhattan $\times$ a detour factor.

Q210. Should restaurant location be treated as an ID? Possibly: restaurant-level target encoding captures average prep time but must be done with CV to avoid leakage.

B4. Time Features Deep Dive

Q211. Why sin/cos encoding for hour? Hour is circular. $\sin(2\pi h/24)$, $\cos(2\pi h/24)$ place 23:00 and 00:00 close together, which linear and distance-based models need. Trees can learn the split anyway.

Q212. Why both sin and cos? Either alone maps two different hours to the same value. Together they identify a unique point on the circle.

Q213. What temporal patterns affect delivery time? Lunch (12–2) and dinner (7–10) peaks, weekends, holidays, festivals, monsoon.

Q214. How did you compute order-to-pickup time? Difference of timestamps in minutes. *(Check this isn't leakage: pickup time is only known after the order is placed.)*

Q215. What is a leakage-prone time feature here? Pickup time, restaurant-ready time, or any timestamp that occurs after the prediction moment.

Q216. How did you handle timezone or format issues? Parsed uniformly with `pd.to_datetime(..., errors='coerce')` and investigated NaT values.

B5. Pipeline and Code-Level Questions

Q217. Show your ColumnTransformer pipeline.

```
from sklearn.compose import ColumnTransformer, make_column_selector as selector
from sklearn.pipeline import Pipeline
from sklearn.impute import KNNImputer, SimpleImputer
from sklearn.preprocessing import StandardScaler, OneHotEncoder

num_pipe = Pipeline([
    ("scale1", StandardScaler()),
```

```

        ("impute", KNNImputer(n_neighbors=5)),
    ])
    cat_pipe = Pipeline([
        ("impute", SimpleImputer(strategy="most_frequent")),
        ("ohe", OneHotEncoder(handle_unknown="ignore")),
    ])
    pre = ColumnTransformer([
        ("num", num_pipe, selector(dtype_include="number")),
        ("cat", cat_pipe, selector(dtype_include="object")),
    ])
    model = Pipeline([("pre", pre), ("est", LGBMRegressor())])

```

Q218. Why use `make_column_selector`? It picks columns by dtype or regex at fit time, so adding a new numeric column doesn't need code edits.

Q219. Danger of `make_column_selector`? Numeric-coded categoricals (like a 0/1 flag or an ID) get treated as numbers. You must cast them explicitly.

Q220. What does `remainder='passthrough'` do? Columns not in any transformer are passed through unchanged instead of dropped (the default is drop).

Q221. Why does `ColumnTransformer` output lose column names, and how do you recover them? It outputs a NumPy array; use `get_feature_names_out()` or `set_output(transform="pandas")`.

Q222. How do you plug the pipeline into cross-validation? Pass the full Pipeline to `cross_val_score`, so the preprocessing is refit on each training fold. This prevents leakage.

Q223. How do you set hyperparameters of a model inside a pipeline? Use the double-underscore syntax, e.g. `est__num_leaves`, or build the pipeline with the Optuna-suggested params.

Q224. How did you handle the pipeline for LightGBM categorical features? Either One-Hot inside the pipeline, or convert to `category` dtype and let LightGBM split natively (needs bypassing One-Hot for those columns).

Q225. How do you save and load the pipeline? `joblib.dump(model, "model.pkl")` and `joblib.load`. Pin library versions, since pickles are version-sensitive.

Q226. Why is pickle risky? Version mismatches break loading, and unpickling untrusted files can execute arbitrary code.

Q227. Show how you'd write the Optuna objective with a pipeline.

```

import optuna
from sklearn.model_selection import cross_val_score

def objective(trial):
    params = {
        "num_leaves": trial.suggest_int("num_leaves", 20, 150),
        "learning_rate": trial.suggest_float("learning_rate", 0.01, 0.3, log=True),
        "n_estimators": trial.suggest_int("n_estimators", 200, 1000),
        "min_child_samples": trial.suggest_int("min_child_samples", 5, 100),
        "subsample": trial.suggest_float("subsample", 0.6, 1.0),
        "colsample_bytree": trial.suggest_float("colsample_bytree", 0.6, 1.0),
        "reg_alpha": trial.suggest_float("reg_alpha", 1e-3, 10, log=True)
    }

```

```

reg_lambda = trial.suggest_float("reg_lambda", 1e-5, 10, log=True),
"reg_lambda": trial.suggest_float("reg_lambda", 1e-3, 10, log=True),
}
pipe = Pipeline([("pre", pre), ("est", LGBMRegressor(**params, random_state=42, verbose=-1))])
scores = cross_val_score(pipe, X_train, y_train, cv=5,
                          scoring="neg_mean_absolute_error", n_jobs=-1)
return -scores.mean()

study = optuna.create_study(direction="minimize", sampler=optuna.samplers.TPESampler(seed=42))
study.optimize(objective, n_trials=100)

```

Q228. Why `direction="minimize"` and the negative sign? sklearn's scorers are "higher is better", so MAE is returned as negative. Negating it gives MAE to minimize.

Q229. Why `subsample` alone may not work in LightGBM? Bagging needs `subsample_freq > 0` for `subsample` to take effect.

Q230. How do you make Optuna reproducible? Seed the sampler, the model, and the CV splitter.

Q231. How would you speed up Optuna? Pruning, fewer folds during search, subsampling data, parallel `n_jobs`, and early stopping inside each trial.

Q232. Can the best CV score be optimistic? Yes. Selecting the best of 100 trials introduces optimistic bias ("winner's curse"). That's why a separate untouched test set is essential.

Q233. Nested CV, when do you need it? When you want an unbiased estimate of the whole tuning procedure: an outer loop for evaluation, an inner loop for tuning. It's expensive, so a held-out test set is the usual compromise.

Q234. What did `study.best_params` give, and how did you use it? Re-trained the final pipeline on the full training set with those params, then evaluated once on the test set.

Q235. How would you visualize the Optuna study? `optuna.visualization.plot_optimization_history`, `plot_param_importances`, `plot_slice`.

Q236. What does `plot_param_importances` tell you? Which hyperparameters most influenced the objective, which helps narrow the search space.

B6. Gradient Boosting Math

Q237. Write the boosting update. $F_m(x) = F_{m-1}(x) + \eta \cdot h_m(x)$, where h_m is a tree fit to the negative gradient of the loss at F_{m-1} .

Q238. What are the pseudo-residuals for squared error? For absolute error? Squared error: $y - F(x)$, the plain residual. Absolute error: $\text{sign}(y - F(x))$.

Q239. XGBoost objective? $\sum L(y_i, \hat{y}_i) + \sum \Omega(f_k)$, with $\Omega(f) = \gamma T + \frac{1}{2}\lambda \sum w_j^2$ (T = number of leaves, w = leaf weights).

Q240. What is the second-order approximation? It expands the loss to second order using gradient g_i and Hessian h_i , which gives an optimal leaf weight $w^* = -G/(H + \lambda)$ and a closed-form split gain.

Q241. Write the split gain. $\text{Gain} = \frac{1}{2} [G_L^2/(H_L + \lambda) + G_R^2/(H_R + \lambda) - (G_L + G_R)^2/(H_L + H_R + \lambda)] - \gamma$.

Q242. What do γ and λ do? γ is the minimum gain required to make a split (pruning). λ shrinks leaf weights (L2).

Q243. What is LightGBM's histogram algorithm? Continuous features are bucketed into ~ 255 bins. Splits are searched over bin boundaries, which cuts memory and time, and the histogram subtraction trick gets a sibling's histogram from the parent for free.

Q244. What is GOSS? Gradient-based One-Side Sampling: keep all large-gradient instances, randomly sample a fraction of small-gradient ones (reweighted), which focuses on poorly fit samples.

Q245. What is EFB? Exclusive Feature Bundling: mutually exclusive sparse features (rarely nonzero together, like One-Hot columns) are bundled into one, cutting feature count.

Q246. Which objective did you use for LightGBM, and should you try others? (*Yours; default is L2.*) MAE target suggests trying `objective='l1'` or `'huber'`, which are more robust to outliers. Note that L1 has a constant gradient magnitude, so convergence can be slower.

Q247. Why not train directly on MAE? Absolute error isn't differentiable at 0 and has zero second derivative, so second-order methods behave poorly. Libraries use workarounds (leaf-value refits with medians). Huber or pseudo-Huber is a smooth compromise.

Q248. What does `min_child_samples` do? Minimum samples in a leaf; larger values prevent fitting tiny, noisy leaves.

Q249. What does `max_bin` do? Number of histogram bins. Fewer bins: faster, more regularized. More bins: finer splits.

Q250. How does `feature_fraction` relate to Random Forest's `max_features`? Both randomly subsample features per tree (or per node) to decorrelate learners.

Q251. How does XGBoost deal with missing values? At each split it learns a default direction for missing values by trying both sides and picking the higher gain.

Q252. How does LightGBM deal with missing values? Similarly, it puts missing values on the side that reduces loss most during split finding.

Q253. XGBoost `tree_method='hist'` vs `'exact'`? Hist uses binned splits (fast); exact enumerates every split (slow but precise).

Q254. CatBoost, did you consider it? It's strong for categorical features (ordered target statistics, symmetric trees). Worth trying here; it's a reasonable extension.

Q255. Why do GBDTs dominate tabular data? They handle mixed types and non-linearities, need little preprocessing, are robust to irrelevant features, and have strong inductive bias for axis-aligned splits.

B7. Linear Models and SVR Math

Q256. Closed-form solution of linear regression? $\beta = (X^T X)^{-1} X^T y$. It fails if $X^T X$ is singular (perfect multicollinearity).

Q257. Closed-form Ridge? $\beta = (X^T X + \lambda I)^{-1} X^T y$. Adding λI makes the matrix always invertible.

Q258. Why does Lasso produce zeros but Ridge doesn't? The L1 constraint region has corners on the axes

Q258. Why does Lasso produce zeros but Ridge doesn't? The L1 constraint region has corners on the axes, so the loss contours tend to hit a corner (a zero coefficient). The L2 ball is smooth, so solutions rarely land exactly on an axis. In terms of gradients, L1's penalty gradient stays constant near zero, pushing small weights to exactly 0.

Q259. Why must you scale before Ridge/Lasso? Penalties apply to coefficient magnitude, which depends on feature units. Unscaled features are penalized unevenly.

Q260. Do you penalize the intercept? No.

Q261. Why didn't Lasso/Ridge improve over OLS much? With a decent sample size and few features the variance is already low; the main error is bias from non-linearity.

Q262. SVR formulation? Minimize $\frac{1}{2}\|w\|^2 + C \sum (\xi_i + \xi_i^*)$ subject to $|y_i - f(x_i)| \leq \epsilon + \xi_i$.

Q263. Role of C? Large C: low tolerance for errors outside the tube (overfitting risk). Small C: more regularization.

Q264. Role of ϵ ? Width of the no-penalty tube. Larger ϵ gives fewer support vectors and a simpler model.

Q265. Role of gamma in RBF? Inverse of the kernel's radius of influence. Large gamma gives wiggly overfit models; small gamma gives smooth, underfit ones.

Q266. What are support vectors? Training points on or outside the ϵ -tube boundaries; only they define the model.

Q267. Why is SVR slow on large data? The kernel matrix is $n \times n$; training complexity is between $O(n^2)$ and $O(n^3)$.

Q268. Could you speed SVR? LinearSVR, subsampling, Nyström approximation, or random Fourier features.

B8. Random Forest Deep Dive

Q269. Why are RF trees decorrelated? Bootstrap sampling plus random feature subsets at every split ensure different trees make different errors.

Q270. Variance of an average of correlated trees? $\text{Var} = p\sigma^2 + (1-p)\sigma^2/B$. Averaging kills the second term; reducing the correlation p (via feature subsampling) lowers the first.

Q271. Why doesn't RF overfit by adding more trees? More trees only reduce variance of the average; the bias stays fixed and error plateaus.

Q272. Does RF reduce bias? No. Deep trees have low bias; RF reduces variance only.

Q273. Mean decrease in impurity vs permutation importance? MDI is computed from training splits and is biased toward high-cardinality features. Permutation is computed on held-out data and is more reliable, but suffers with correlated features.

Q274. Why is RF's extrapolation poor? Predictions are averages of training targets, so they can never exceed the training range.

Q275. max_features default for regression in sklearn? 1.0 (all features) in recent versions, which makes it closer to bagging; tuning it lower often helps.

Q276. RF vs ExtraTrees? ExtraTrees picks random thresholds and usually doesn't bootstrap; it's faster and has lower variance but more bias.

B9. Evaluation and Statistics Deep Dive

Q277. Formula for R^2 . $R^2 = 1 - \frac{\sum(y - \hat{y})^2}{\sum(y - \bar{y})^2}$.

Q278. Can R^2 on the test set differ from CV R^2 ? Yes. R^2 depends on the target variance of that sample; a test set with lower variance yields lower R^2 even at the same error.

Q279. Why is R^2 not comparable across datasets? It depends on target variance. MAE in minutes is comparable.

Q280. How would you build a confidence interval for MAE? Bootstrap the test predictions, 1,000 resamples, report the 2.5 and 97.5 percentiles.

Q281. How would you test LightGBM vs RF formally? Paired bootstrap on per-sample absolute errors, or a paired Wilcoxon on fold-wise MAEs from repeated CV. A standard t-test across folds is biased because folds overlap in training data (Nadeau–Bengio correction exists).

Q282. Heteroscedasticity in this problem? Yes. Errors grow with distance and time, since long trips have more variance. It affects interval estimates and OLS inference.

Q283. What would you do about heteroscedasticity? Log-transform the target, model the variance separately, or use quantile regression.

Q284. Median vs mean prediction? Minimizing MAE targets the conditional median; minimizing MSE targets the conditional mean. For skewed delivery time they differ.

Q285. Why might median predictions suit ETA? They're robust to long-tail delays and typically closer to the "typical" experience.

Q286. Calibration of prediction intervals? For a 90% interval, ~90% of actuals should fall inside. Check empirical coverage on a validation set.

Q287. Pinball loss? Quantile loss: $\max(q \cdot e, (q-1) \cdot e)$ with $e = y - \hat{y}$.

Q288. How would you evaluate by segment? Group by distance buckets, hour, city, and compute MAE per group; report the worst segments.

Q289. What is "Simpson's paradox" risk here? An aggregate improvement hiding degraded performance in important segments.

Q290. What baseline would be most convincing? Per-restaurant-zone historical median delivery time, which is what a simple ops heuristic would use.

B10. Overfitting, Generalization, Leakage Deep Dive

Q291. Your train MAE was X and test MAE 3.01. What gap would worry you? A train MAE far below test (e.g. 1.0 vs 3.0) signals overfitting. Some gap is normal for boosting.

Q292. Name five sources of leakage in this project. Imputer/scaler fit on full data; target encoding without

CV; features using post-order information; duplicate or near-duplicate rows across train and test; hyperparameter tuning on the test set.

Q293. How would duplicates cause leakage? Identical or nearly identical orders land in both splits, so the model memorizes them and test scores inflate.

Q294. How would you check for train-test contamination? Check duplicate rows across splits, compare distributions (adversarial validation).

Q295. What is adversarial validation? Train a classifier to distinguish train from test. If it separates them well (high AUC), the distributions differ and the test isn't representative.

Q296. Group leakage? If the same restaurant or rider appears in both train and test and the model memorizes ID-level behavior, scores are optimistic compared with new restaurants. Use GroupKFold to test generalization to unseen groups.

Q297. When should you use GroupKFold? When samples within a group are correlated (same restaurant, rider, or customer) and you want to generalize to new groups.

Q298. What is concept drift here? Relationships change over time, such as new traffic rules, monsoon, platform policy changes, and festival seasons.

Q299. What is covariate shift? Input distribution changes while $P(y|x)$ stays the same, such as expansion into new cities.

Q300. How would you detect drift in production? PSI or KS test per feature, monitoring prediction distribution and rolling error once ground truth arrives.

B11. Feature Importance and Explainability Deep Dive

Q301. LightGBM importance types? `split` (number of times used) and `gain` (total loss reduction). Gain is generally more informative.

Q302. SHAP vs permutation importance? SHAP gives local, additive attributions consistent with the model; permutation measures global impact on score. SHAP TreeExplainer is exact and fast for trees.

Q303. How would you explain a single prediction to a customer-ops team? A SHAP waterfall plot showing how distance, hour, and rider rating push the ETA up or down.

Q304. What does a SHAP dependence plot of distance show? The marginal effect of distance on predicted time, revealing non-linear shape and interactions.

Q305. Why are correlated features a problem for importance? Importance is split or shared between them, so each looks less important than it really is.

Q306. Partial dependence vs SHAP dependence? PDP averages over the data, assuming feature independence. SHAP dependence shows per-sample variation.

Q307. Does feature importance imply causality? No. It's predictive association only.

B12. Production and System Design

Q308. Design the ETA system end to end. Order placed → feature service (distance, time, rider and restaurant stats) → model service → ETA returned to the app. A batch job retrains weekly, logs predictions and actuals, and a monitoring dashboard tracks MAE and drift.

Q309. Online vs batch inference? ETA needs online (real-time) inference at order time, plus possibly updates as the order progresses.

Q310. Latency budget? Typically tens of milliseconds. LightGBM predicts in about a millisecond per row, so feature retrieval dominates.

Q311. How would you handle feature freshness (traffic)? A feature store with streaming updates, and consistent definitions for train and serve.

Q312. Should ETA be static or updated? Updated at order stages (accepted, picked up, en route) with stage-specific models, which are much more accurate than one at order time.

Q313. How would you A/B test the model? Randomize users or orders to old vs new ETA, measure MAE, customer cancellations, late-delivery rate, and ratings. Check the sample size and run long enough to cover weekly seasonality.

Q314. Metric to optimize business-wise? Not just MAE. The share of orders delivered later than the promised ETA matters more to customers, so a slightly conservative (upper-quantile) ETA may be better.

Q315. Feedback loop risk? If the displayed ETA influences rider or customer behavior, future training data is affected by the model itself.

Q316. How to handle model versioning? MLflow or similar for experiment tracking, versioned artifacts, and rollback.

Q317. How do you test the pipeline code? Unit tests for feature functions (like Haversine on known city pairs), schema validation, and a regression test with a golden set.

Q318. Known test case for Haversine? Delhi (28.6139, 77.2090) to Mumbai (19.0760, 72.8777) is about 1,150 km.

Q319. What if lat/long is missing at inference? Fallback to a default model without distance, or reject and request a valid address.

Q320. Privacy concerns? Customer addresses and rider data are personal; anonymize, minimize retention, and follow data-protection regulations.

B13. Scenario-Based Questions

Q321. MAE is great on average but customers complain about late deliveries. Why? Average error hides tails. Under-predictions in peak hours or bad weather hurt. Check the late-delivery rate and error by segment, and consider upper-quantile predictions.

Q322. After deployment, MAE doubles. Debug steps? Check data pipeline and schema changes, feature distribution drift, new cities or restaurants, upstream bugs (units, null encoding), and train-serve skew, before suspecting the model.

Q323. A new feature improves CV MAE slightly but hurts the test set. What do you do? Suspect noise or

overfitting to CV. Check significance across repeated CV, and don't ship on a small unstable gain.

Q324. The business wants the model to explain why an ETA is long. What do you offer? SHAP-based reason codes: "long distance", "dinner peak", "heavy rain".

Q325. The model must run on a phone, no server. What changes? Compress it: fewer trees, quantization, or convert to ONNX or a simple lookup, and drop heavy preprocessing like KNN imputation.

Q326. Only 5% of the data is labeled with accurate delivery time. Options? Semi-supervised or weak-supervision approaches, active collection, and careful validation on the clean subset.

Q327. Data has 10 million rows. What changes? Replace KNN imputer, use LightGBM's histogram method, subsample for tuning, and use Dask or Spark for preprocessing.

Q328. A stakeholder says "just use deep learning". Response? I'd benchmark an MLP or TabNet against LightGBM; for tabular data of this size boosting usually wins on accuracy, speed, and interpretability, but I'd test it, not assume.

Q329. How do you handle festival days with unusual demand? Add holiday and event flags, and monitor errors on those days.

Q330. Rider ratings are missing for new riders. Strategy? Impute with a global prior, add a "new rider" flag, and avoid KNN imputing from unrelated riders.

Q331. Two features are 95% correlated. Do you drop one? For trees, not necessary; for linear models, drop one or use Ridge. For interpretability, drop one.

Q332. A test-set record has delivery time of 3 minutes for 12 km. What do you do? Flag it as a data error, apply plausibility rules (speed bounds), and exclude from train and test with a documented rule.

Q333. Should outliers in the target be removed from the test set? Only with a defensible rule applied equally to both sets; otherwise you inflate performance.

B14. Tough Pushback Questions (practice staying calm)

Q334. "Your MAE of 3 minutes sounds too good. What's wrong?" Possibly the dataset is cleaner than reality (synthetic or curated), or there's leakage. I checked that all features are available at order time and that preprocessing was fit on train only. I'd also say this is a public-style dataset, so I wouldn't promise the same on production data.

Q335. "Isn't this just a Kaggle project?" The dataset may be public, but the work (cleaning, leakage-free pipelines, model comparison, tuning, error analysis) follows the same workflow as a production project. I'm clear about the gap to production: no traffic data, no deployment, no monitoring.

Q336. "What did you personally contribute versus copying notebooks?" Be specific: the imputation choice and its reasoning, the feature set, the pipeline structure, the Optuna search space, and the ablations you ran. If you borrowed ideas, say which and how you verified them.

Q337. "Why should I believe LightGBM is truly better than RF?" Gap is 0.23 min MAE and 0.03 R^2 . I'd back it with fold-wise results and a paired test. Even if the two are close, LightGBM also trains and predicts faster.

Q338. "Your R^2 would be different on another random seed. How different?" Probably within ± 0.01 . I'd

verify by repeating the split with multiple seeds and reporting mean $\pm$ std.

Q339. "What's the weakest part of your project?" The random split and the lack of road-network and traffic features. I'd rather name them honestly and explain the fix.

Q340. "Why didn't you try neural networks?" Scope and time; for this tabular dataset boosting is the standard strong baseline. It's a reasonable follow-up.

Q341. "Explain LightGBM to a non-technical person." Imagine many small decision rules, each one fixing the mistakes of the ones before it. Together they predict delivery time from distance, time of day, and order details.

Q342. "What if the mentor's feedback contradicted your approach?" Test both empirically, then decide on evidence and document the result.

Q343. "Did you try anything that failed?" Have one real example ready, such as a feature that didn't help, SVR's poor performance, or a hyperparameter range that overfit. Interviewers value honest failures.

B15. Behavioral and Internship Questions

Q344. Why did you choose this project? Delivery ETA is a real-world regression problem with business impact, touching the full ML workflow.

Q345. How did you work with your mentor? Regular check-ins, shared results and decisions, applying feedback. Give one concrete example of feedback you acted on.

Q346. A time you were stuck? Describe something real, such as handling 17% missing data or tuning instability, and how you resolved it.

Q347. How did you manage your time over the internship? Split into EDA, preprocessing, modeling, tuning, and reporting, with milestones.

Q348. What would you do in the first 30 days of a new data science role? Learn the data and metrics, reproduce the existing baseline, find quick wins in error analysis.

Q349. What feedback did you receive? (*Real answer.*) For example: add ablations, avoid leakage, report the baseline clearly.

Q350. What are you proudest of? The leakage-free pipeline structure and a systematic, fair model comparison.

B16. Rapid-Fire Round

Question	Short answer
What's <code>n_jobs=-1</code> ?	Use all CPU cores
<code>cross_val_score</code> default for regressors?	R^2
Why <code>neg_mean_absolute_error</code> ?	sklearn maximizes scores, so errors are negated

Is KNN a lazy learner?	Yes, no training phase, computation happens at prediction
Default <code>k</code> in <code>KNNImputer</code> ?	5
Default kernel for SVR?	RBF
LightGBM default <code>num_leaves</code> ?	31
LightGBM default <code>learning_rate</code> ?	0.1
XGBoost default <code>max_depth</code> ?	6
RF default <code>n_estimators</code> (sklearn)?	100
Is LightGBM sensitive to scaling?	No
Does Lasso need scaling?	Yes
What does <code>handle_unknown='ignore'</code> do?	Encodes unseen categories as all-zeros
Which split ensures reproducibility?	Setting <code>random_state</code>
Haversine output unit with <code>R = 6371</code> ?	km
MAE sensitive to outliers?	Less than RMSE
Boosting reduces what?	Bias (and some variance with shrinkage)
Bagging reduces what?	Variance
What's early stopping's monitored set?	A validation set, never the test set
Optuna default sampler?	TPE

PART C: PREPARATION CHECKLIST

1. **Know your own numbers cold:** dataset size, number of features, target mean and std, `k` for KNN, number of CV folds, number of Optuna trials, best hyperparameters, and the full results table for all 7 models.
2. **Be ready to open the notebook** and explain every cell, especially preprocessing and the split order.
3. **Prepare two honest weaknesses** (random split, no traffic data, no deployment) and the fix for each.
4. **Prepare one failure story** and one decision you changed based on data.
5. **Re-derive from memory:** Haversine, R^2 , the 36% calculation, KNN imputation steps.
6. **Practice saying the 1-minute pitch** in under 60 seconds, then the 3-minute version with technical detail.
7. **Do not claim anything you did not do.** Every (*your value*) in this doc is a place where an interviewer will probe.